EXPERIMENT NO. 3

CAPTURING AND ANALYZING HTTP, FTP, AND TELNET TRAFFIC USING WIRESHARK

AIM:

To capture live network traffic generated by HTTP, FTP, and Telnet services using Wireshark, reconstruct the underlying TCP conversations, and examine the unencrypted exchange in order to understand how each protocol operates and how login credentials can be exposed in cleartext.

OBJECTIVES:

1. To understand the fundamentals of packet capturing and protocol analysis using Wireshark.
2. To capture and examine HTTP request-response pairs exchanged between a browser and a web server.
3. To capture an FTP login session and observe how credentials are transmitted.
4. To capture a Telnet session and analyze the plaintext exchange of login credentials and commands.
5. To evaluate the security implications of using unencrypted network protocols.

HARDWARE & SOFTWARE REQUIREMENTS:

Category	Specification & Tools
Hardware Configuration	Laptop/PC with Intel Core i5 (8th Gen or higher), minimum 8 GB RAM, active internet connection
Operating System	Windows 11
Network	Active Wi-Fi/LAN Connection
Web Server	XAMPP Apache Server
FTP Server	Microsoft FTP Service
Telnet Server	KpyM Telnet Server
Packet Analyzer	Wireshark
Browser	Microsoft Edge
Command Line Tool	Windows Command Prompt
Documentation Tools	Microsoft Word

THEORY:

Network packet analysis is the practice of intercepting and inspecting the data units exchanged between two or more networked devices. It plays a key role in understanding client-server communication, diagnosing network faults, and uncovering security weaknesses in a network.

Wireshark is a widely used open-source protocol analyzer that captures packets traversing a network interface and displays them in a readable, layered format. It supports analysis of a very large number of protocols at the packet level.

HTTP (HyperText Transfer Protocol) governs communication between web browsers and web servers. Because HTTP does not encrypt its payload by default, the complete request and response can be read directly from a capture.

FTP (File Transfer Protocol) is used to move files between a client and a server. The classic implementation of FTP sends the username and password as plaintext, so both can be recovered simply by inspecting the captured packets.

Telnet provides remote command-line access to another machine over the network, but like FTP it does not encrypt anything it sends, exposing login credentials and every command typed during the session.

Wireshark's Follow TCP Stream option stitches together all the packets belonging to one TCP conversation, letting the analyst view the exchange the way the two endpoints actually sent and received it.

PACKET CAPTURE & TRAFFIC ANALYSIS – PHASES, TOOLS & COMMANDS:

1. HTTP Traffic Capture

Wireshark captures the HTTP packets generated when a webpage hosted on the local Apache server is accessed. The capture is then reviewed to study the HTTP request and response.

Tool / Utility	Purpose	Command Used
XAMPP	Host HTTP server	Start Apache
Edge	Generate HTTP traffic	http://localhost
Wireshark	Capture HTTP packets	Filter: http

2. HTTP TCP Stream Analysis

The captured HTTP packets are reassembled with Follow TCP Stream to view the full HTTP conversation between client and server.

Tool / Utility	Purpose	Command Used
Wireshark	Reassemble HTTP conversation	Follow TCP Stream

3. FTP Credential Capture

An FTP session is started from the Windows command line to the local FTP server. Wireshark captures the control-channel packets exchanged during authentication, so the transmitted username and password can be viewed in plaintext.

Tool / Utility	Purpose	Command Used
Microsoft FTP Service	FTP server	Start FTP service
Command Prompt	Connect to FTP	ftp localhost
Wireshark	Capture FTP packets	Filter: ftp

4. Telnet Credential Capture

A Telnet session is established with the local Telnet server. Wireshark captures the login exchange and the commands issued afterward, allowing the plaintext username and password to be analyzed.

Tool / Utility	Purpose	Command / Action Used
KpyM Telnet Server	Telnet server	Start service
Command Prompt	Connect to Telnet	telnet localhost
Wireshark	Capture Telnet packets	Filter: telnet

5. TCP Stream Analysis

The HTTP, FTP, and Telnet packets captured earlier are reassembled with Follow TCP Stream to reconstruct the full client-server communication and confirm that sensitive information is sent without encryption.

Tool / Utility	Purpose	Command / Action Used
Wireshark	View complete communication	Follow TCP Stream

IMPLEMENTATION STEPS:

1. HTTP Traffic Capture

- Started Apache through the XAMPP control panel.
- Confirmed the server was running by loading http://localhost in Edge.
- Launched Wireshark and began capturing on the active Wi-Fi adapter.
- Applied the http display filter to isolate HTTP packets.
- Reloaded the page to generate fresh request/response traffic.
- Used Follow TCP Stream to view the reconstructed HTTP conversation.

2. FTP Credential Capture

- Installed and started Microsoft FTP Service, and created a local FTP account (student).
- Connected to the server from the command prompt using ftp localhost.
- Authenticated using the configured username and password.
- Executed the dir command to generate additional FTP traffic.
- Captured the session in Wireshark using the loopback adapter.
- Used Follow TCP Stream to inspect the exchanged USER and PASS commands.

3. Telnet Credential Capture

- Installed and started the KpyM Telnet Server, and confirmed port 23 was listening using netstat.
- Connected using telnet localhost from the command prompt.
- Logged in with the configured credentials and ran a few basic commands (dir, ipconfig).
- Captured the session in Wireshark using the loopback adapter.
- Used Follow TCP Stream to reconstruct the login and command sequence.

4. TCP Stream Reassembly and Analysis

- Reassembled all three captured sessions (HTTP, FTP, Telnet) using Follow TCP Stream.

- Reviewed each reconstructed conversation for its application-layer content.
- Confirmed that FTP and Telnet exposed the username and password in plaintext.
- Documented the findings for the security assessment.

OBSERVATIONS:

Live HTTP, FTP, and Telnet traffic was captured with Wireshark over the course of the practical, and each TCP conversation was reassembled to inspect the application-layer content exchanged between client and server.

- The HTTP capture showed the complete GET request and the corresponding 200 OK response between Edge and the XAMPP Apache instance.
- The FTP session revealed the login username (student) and password (student123) in plaintext when the stream was followed.
- The Telnet capture displayed the full login sequence — username, password, and the commands typed afterward — with no encryption applied.
- Reassembling all three streams confirmed that HTTP, FTP, and Telnet expose meaningful data, and in the case of FTP and Telnet, login credentials, to anyone able to observe the traffic.

OUTPUT:

Phase 1 – HTTP

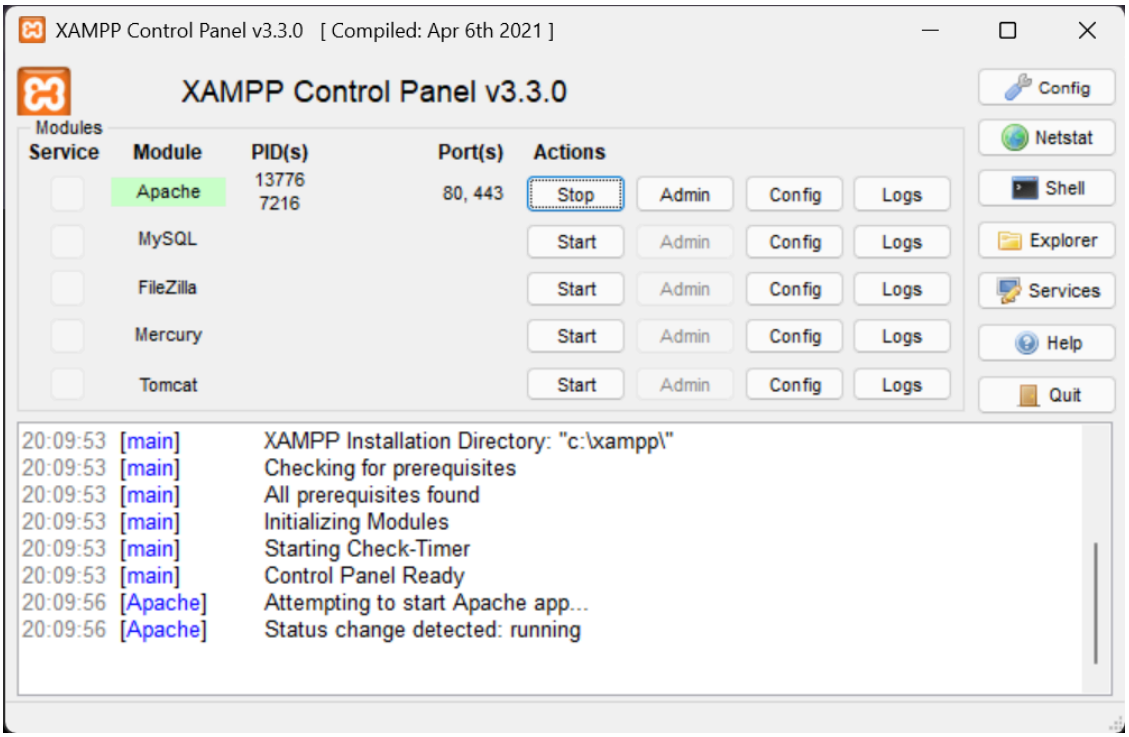


Fig. 1: Apache Server Running (XAMPP)

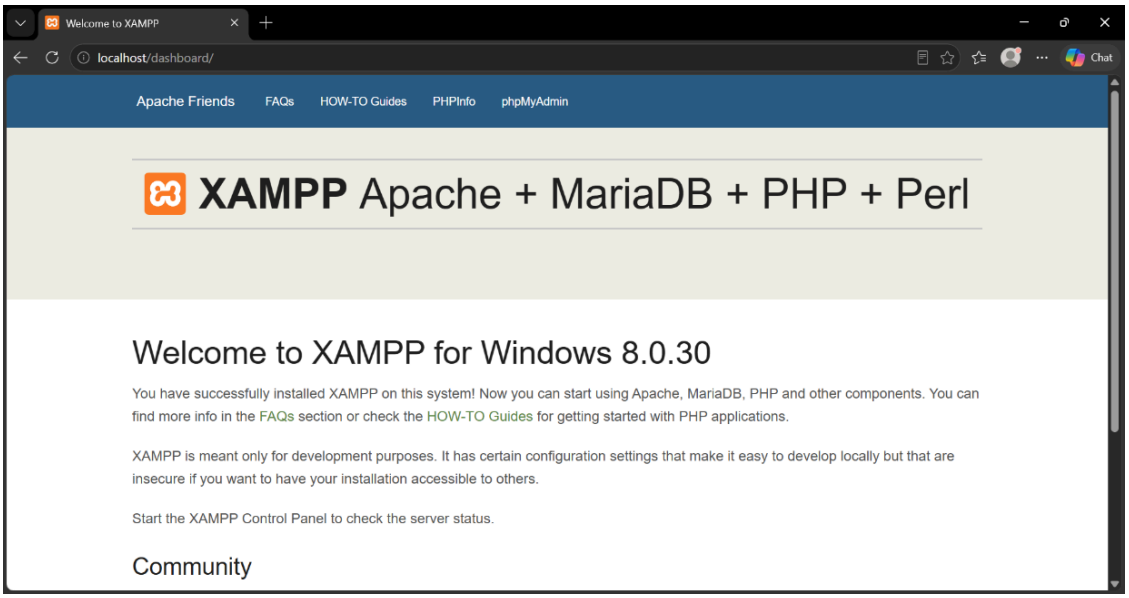


Fig. 2: Localhost Dashboard

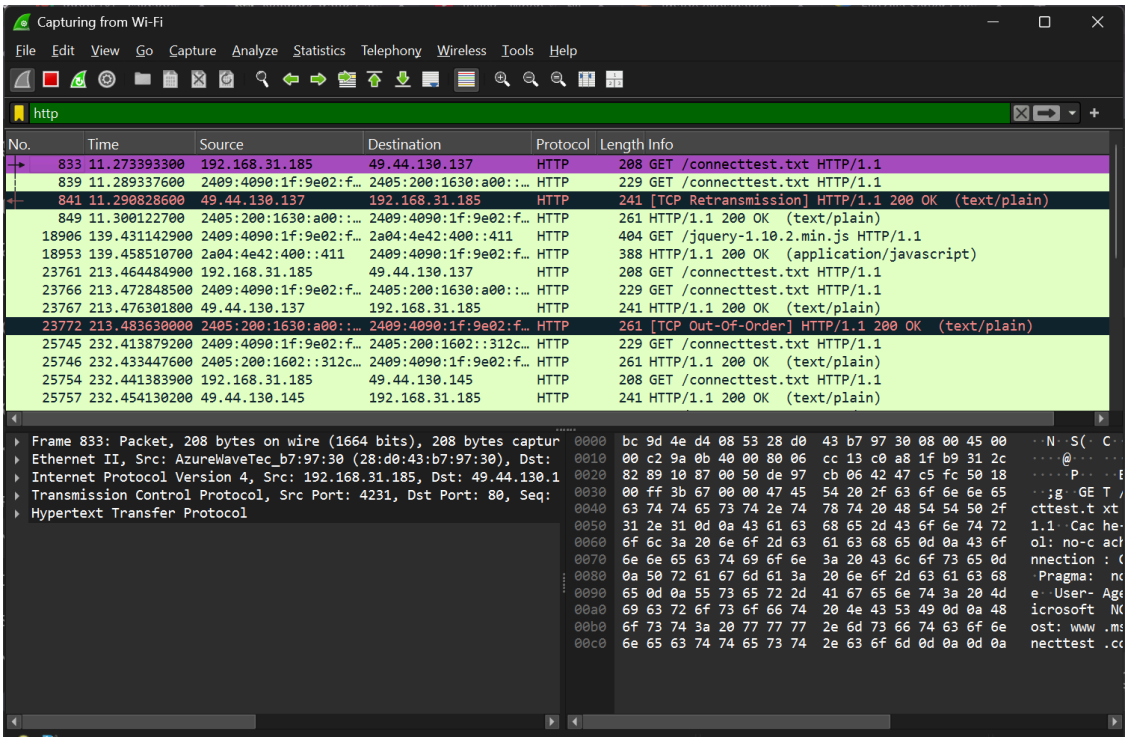


Fig. 3: HTTP Packet Capture in Wireshark

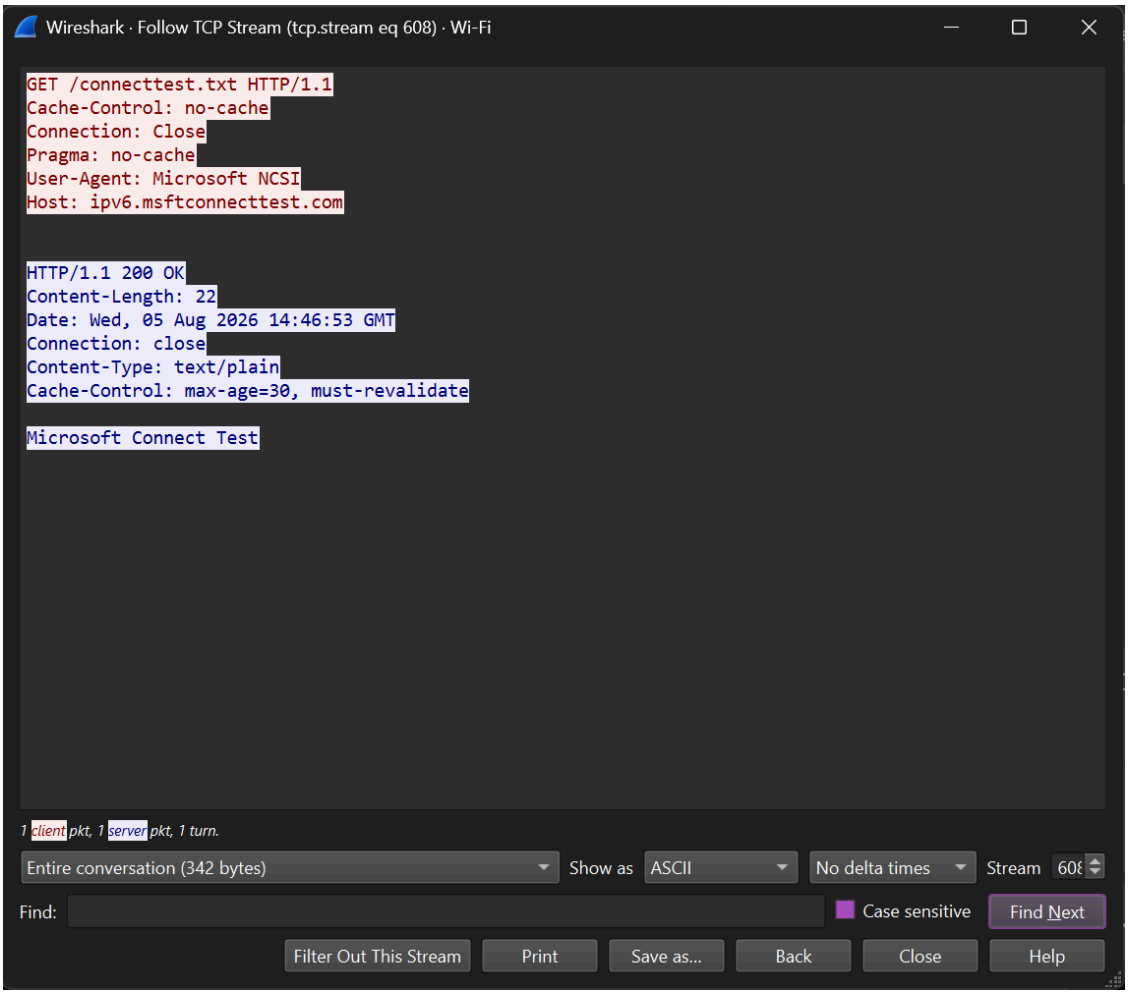


Fig. 4: HTTP Follow TCP Stream

Phase 2 – FTP

```
C:\Users\salga>ftp localhost
Connected to Pooh.
220 Microsoft FTP Service
200 OPTS UTF8 command successful - UTF8 encoding now ON.
User (Pooh:(none)): student
331 Password required
Password:

230 User logged in.
ftp> dir
200 EPRT command successful.
125 Data connection already open; Transfer starting.
08-05-26  08:27PM                   20 text.txt
226 Transfer complete.
ftp: 52 bytes received in 0.00Seconds 17.33Kbytes/sec.
ftp> bye
221 Goodbye.
```

Fig. 5: FTP Login using Command Prompt

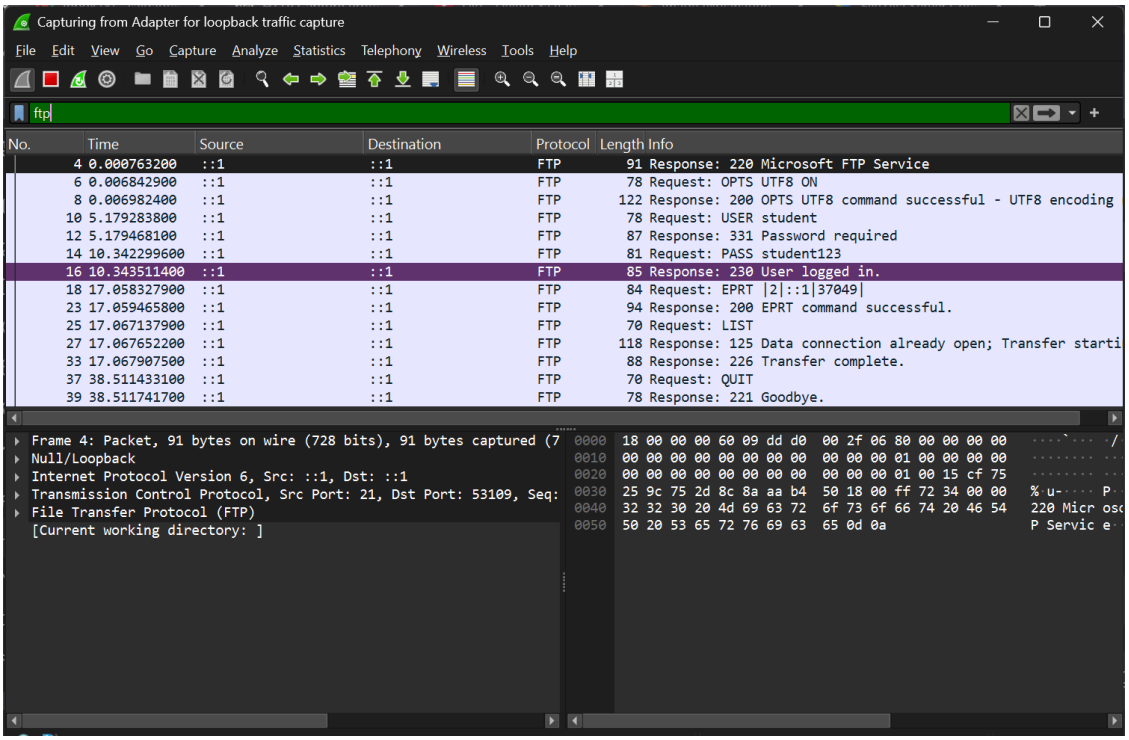


Fig. 6: FTP Packet Capture in Wireshark

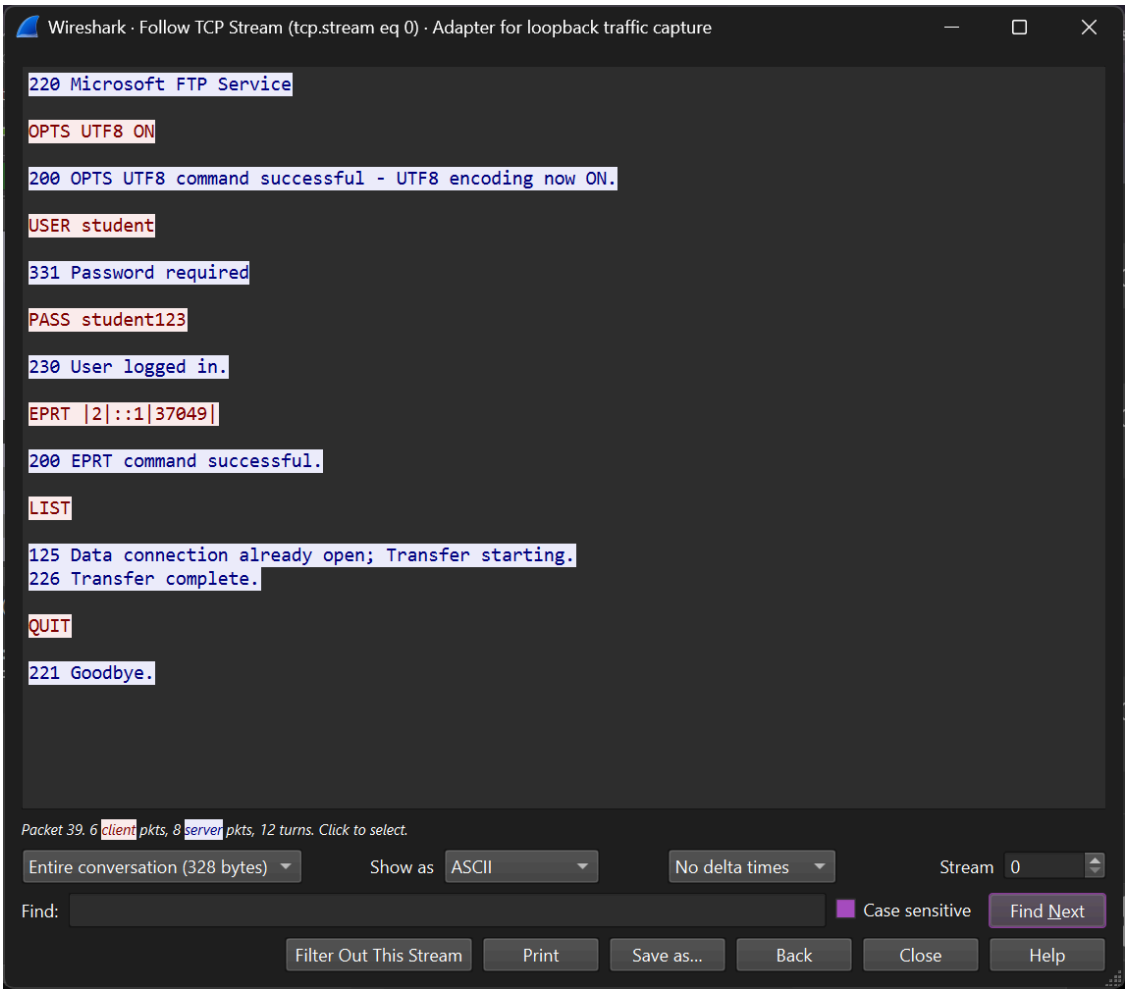


Fig. 7: FTP Follow TCP Stream Showing USER and PASS

Phase 3 – Telnet



Fig. 8: Port 23 Verification using netstat

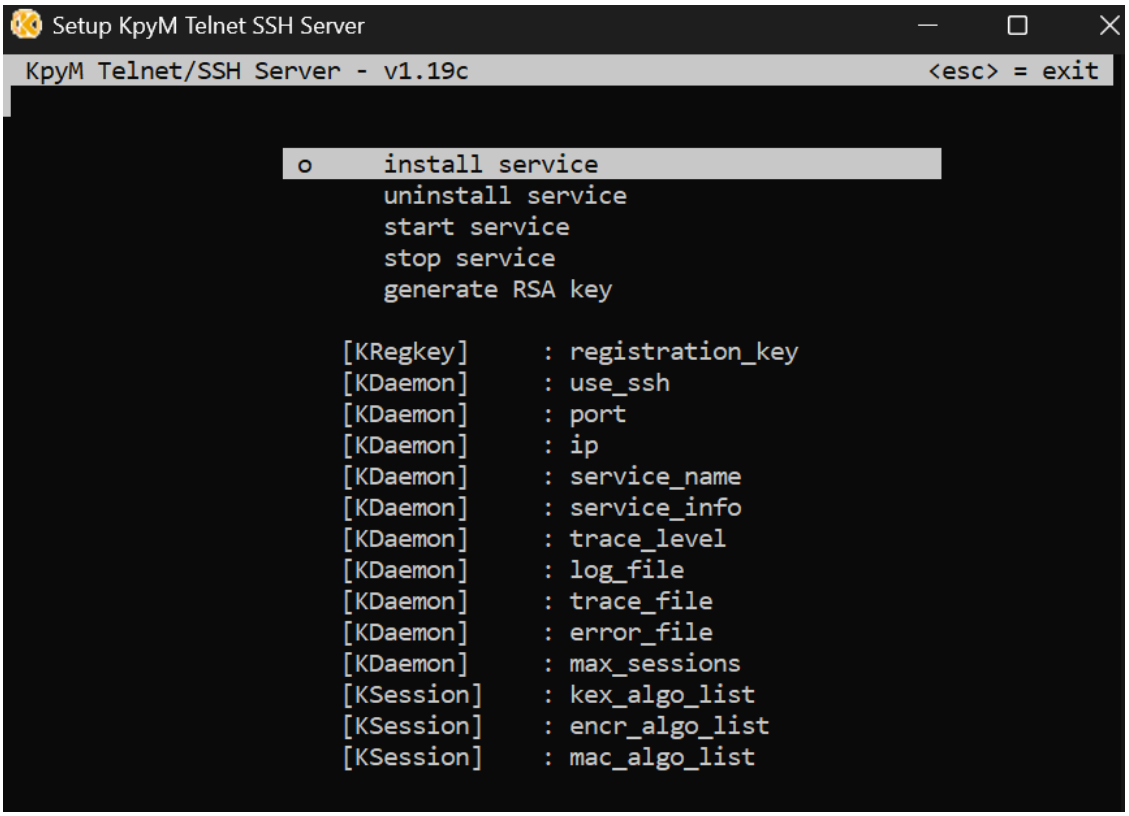


Fig. 9: KpyM Telnet Server Running

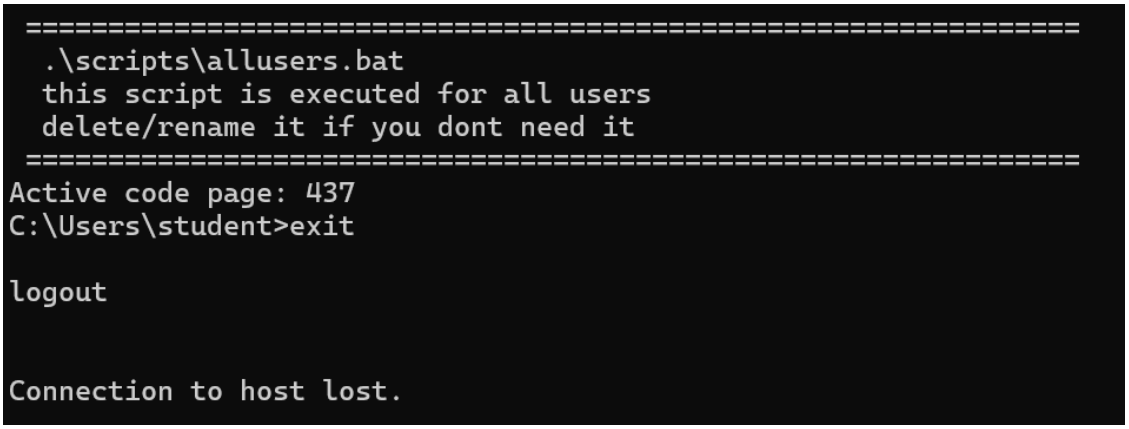


Fig. 10: Telnet Login

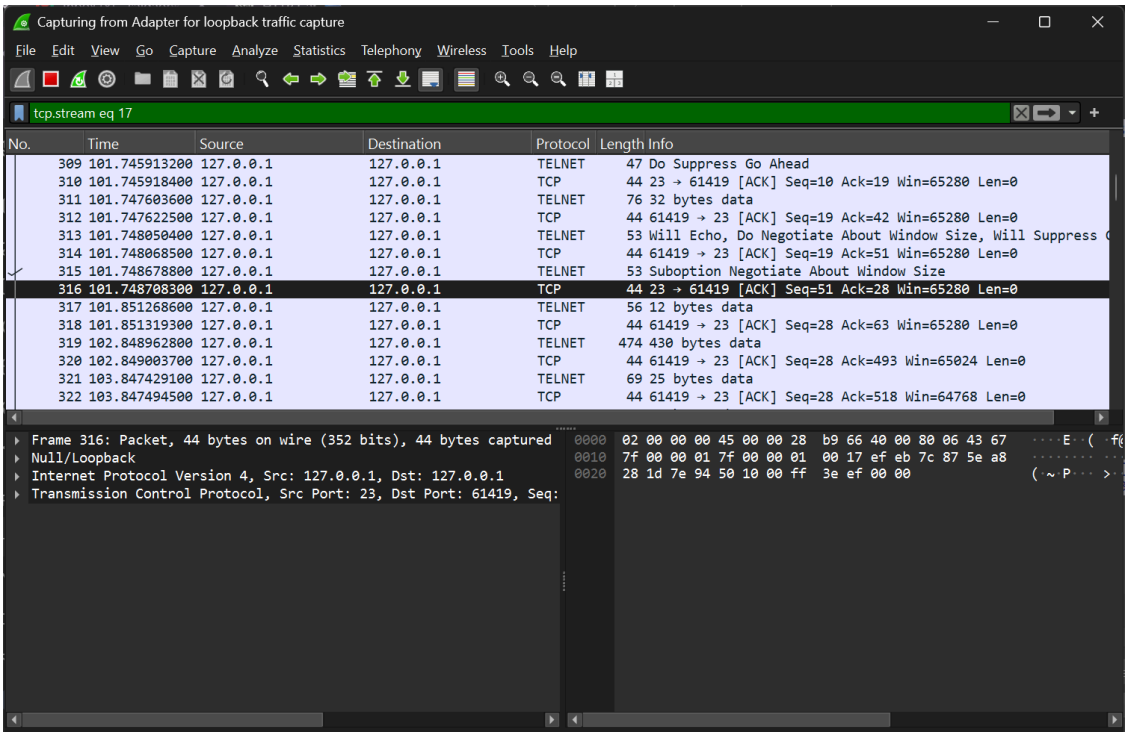


Fig. 11: Telnet Packet Capture

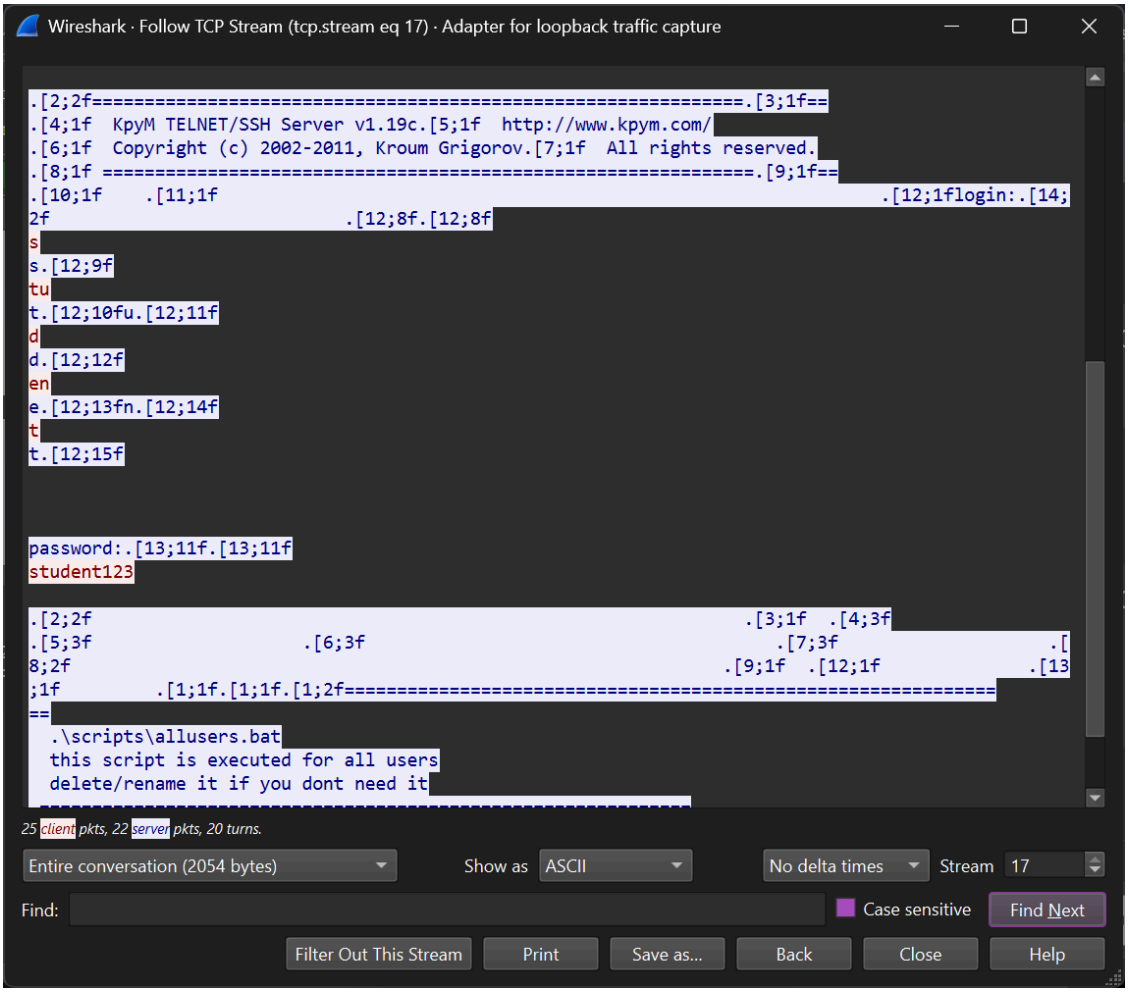


Fig. 12: Telnet Follow TCP Stream Showing Username and Password

ANALYSIS:

The exercise successfully captured and analyzed live HTTP, FTP, and Telnet traffic with Wireshark. The HTTP capture showed the full exchange between the browser and the Apache server, while the FTP and Telnet sessions confirmed that both protocols transmit login credentials without any encryption. Reconstructing each TCP stream made it straightforward to read exactly what was sent between client and server, underlining the risk of running these protocols on an untrusted network.

PREVENTIVE MEASURES:

1. Replace HTTP, FTP, and Telnet with their encrypted counterparts — HTTPS, SFTP/FTPS, and SSH — wherever possible.
2. Enforce strong authentication and encrypt traffic end-to-end to keep credentials and data confidential.
3. Periodically audit network traffic and keep server software patched to close known vulnerabilities.
4. Disable legacy plaintext services on production systems unless they are strictly required and isolated.

CONCLUSION:

This experiment demonstrated packet capture and protocol analysis using Wireshark across HTTP, FTP, and Telnet traffic. Reconstructing the TCP streams confirmed that FTP and Telnet transmit login credentials in plaintext, reinforcing the importance of adopting secure alternatives such as HTTPS, SFTP, and SSH in place of these legacy protocols.